

# Final Report: Design, Build, and Evaluation of a Cloud-Native Multimodal AI Assistant

## Abstract

This project presents a cloud-native AI assistant deployed on GCP to support planning, research, and task automation. The system combines a large language model (Gemini), speech services for voice input/output, and a tool-calling layer that can access web APIs, cloud functions, a database, and sandboxed shell commands. The assistant is implemented as a FastAPI backend on Cloud Run, with optional hybrid local-client usage. Evaluation on practical test tasks shows the assistant improves speed and convenience for common study and planning workflows compared with manual, non-AI methods.

## 1. Research Process and Tools Explored

The research process focused on selecting a lightweight architecture that still satisfies real cloud AI requirements.

Tools and options explored: - LLM providers: Gemini API as primary model due to GCP alignment and strong multimodal roadmap. - Backend frameworks: FastAPI selected for low-latency API development and simple deployment on Cloud Run. - Speech stack: Cloud Speech-to-Text and Cloud Text-to-Speech selected for native GCP integration. - Storage options: Firestore chosen for schema-flexible, serverless persistence. - Orchestration options: direct in-process tool router with optional Cloud Function endpoint for external actions. - CI/CD and deployment: Cloud Build + Artifact Registry + Cloud Run.

Selection rationale: - Minimal operational overhead. - Strong cloud-native fit. - Easy extensibility for additional tools and modalities.

## 2. Technical Design

### 2a. High-Level Architecture Diagram

Mermaid diagram is provided in docs/architecture.mmd.

### 2b. Cloud Services Used

- Cloud Run: hosts assistant API service.
- Cloud Functions: executes external automation actions via authenticated HTTP.
- Firestore: stores and retrieves assistant facts/notes and persistent chat history (user + assistant turns).
- Speech-to-Text: transcribes uploaded voice audio.
- Text-to-Speech: converts generated text to audio responses.
- Cloud Build and Artifact Registry: build and publish deployable container images.

## 2c. Model(s) Used and Why

Primary model: - Gemini via Vertex AI (gemini-2.5-flash default in deployment): selected for low latency, strong instruction-following, and direct GCP integration.

Optional fallback: - OpenAI model configured only as backup in hybrid/cross-provider tests.

Why chosen: - Primary path remains fully GCP-aligned. - Fallback improves resilience for cross-cloud interoperability experiments.

## 2d. Data Flow and User Interaction Flow

1. User sends text, voice, or image input to Cloud Run API.
2. Input passes security checks (length limits, sanitization, token checks where enabled).
3. Assistant planner decides whether a tool call is needed.
4. Tool executes (web search, Firestore lookup, Cloud Function action, allow-listed shell command, or allow-listed external API).
5. LLM generates final response based on user query plus tool output.
6. Response is returned as text and voice-capable output; the deployed homepage plays synthesized audio replies in addition to showing text.

Homepage multimodal UX: - Single adaptive action button supports both modalities. - Empty textbox: button acts as microphone start/end toggle. - After capture, transcript fills textbox and button switches to send mode. - Non-empty textbox: same button sends text. - Homepage loads recent Firestore history for the user id so past conversation appears on refresh.

Practical use case demonstrated: - A student asks for a weekly study plan and requests fresh topic updates. The assistant fetches updates via tool-calling, summarizes them, and returns a structured plan.

## 2e. API Summary Used

Core assistant APIs (FastAPI endpoints): - GET /: serves the web UI. - POST /assistant/chat: text chat with optional tool-calling. - POST /assistant/voice/chat: end-to-end voice chat (transcribe + answer + synthesize). - POST /assistant/voice/transcribe: speech-to-text only. - POST /assistant/voice/synthesize: text-to-speech only. - POST /assistant/document/analyze: uploaded document summary/extraction. - GET /assistant/conversations: list conversation summaries. - GET /assistant/conversations/{session\_id}: load one conversation thread. - GET /assistant/history: retrieve recent stored messages.

External/cloud APIs and services integrated: - Vertex AI Gemini API (primary LLM inference). - Google Cloud Speech-to-Text API (audio transcription). - Google Cloud Text-to-Speech API (audio synthesis). - Firestore API (chat and conversation persistence). - Optional Brave Search API via web\_search

tool. - Optional Cloud Function HTTP endpoint via `call_cloud_function` tool. - Optional allow-listed external HTTP APIs via `http_api` tool.

### 3. Performance Evaluation

#### 3a. Supported Task Types

- Study planning and scheduling.
- Short research summarization from web results.
- Basic automation via Cloud Function actions.
- Voice-input conversation and speech synthesis playback in the same chat flow.

#### 3b. Accuracy and Utility

Evaluation method: - Defined prompt set in `evaluation/test_cases.json`. - Automatic keyword coverage scoring in `evaluation/run_eval.py`. - Human review for relevance, structure, and actionability.

External tools available for use: - `call_cloud_function`: can be used for automation actions through the configured function router. - `web_search`: can be used for fresh web retrieval when API key and quota are available. - `firestore_lookup` (tool): can be used to retrieve app-stored facts/documents. - `shell_command`: can be used in sandbox mode with a strict allow-list. - `http_api`: can be used for allow-listed external API calls.

Observed results: - High utility in structured planning prompts. - Good summarization quality when tool results are available. - Automation responses were reliable for predefined actions.

#### 3c. Limitations and Failure Modes

- Tool-selection JSON may fail under ambiguous prompts.
- Web-search quality depends on external API quality and quota.
- Shell tool intentionally constrained; cannot perform complex commands.
- Voice pipeline currently assumes standard audio format parameters.

#### 3d. Comparison to Non-AI Workflow

Without assistant: - User manually searches multiple websites, organizes notes, and executes commands separately.

With assistant: - Single interaction pipeline combines retrieval, synthesis, and action recommendation. - Lower task-switching overhead. - Faster first-draft planning and research summaries.

## 4. Challenges and Solutions

Challenge 1: Securely exposing tool execution. - Solution: strict command allow-list and syntax blocking for shell tool.

Challenge 2: Preventing API credential exposure. - Solution: environment-based secret injection, no hard-coded keys, and deployment-time variable controls.

Challenge 3: Keeping architecture simple but extensible. - Solution: modular service boundaries (assistant logic, tool registry, security module, deployment scripts).

## 5. Key Learnings About Cloud AI Systems

- Cloud-native AI assistants require both model quality and robust surrounding systems.
- Tool-calling substantially improves practical utility beyond text generation alone.
- Security controls are not optional; they must be designed early in architecture.
- Serverless components reduce maintenance burden while supporting fast iteration.

## 6. Future Improvements

- Native Gemini multimodal image caption integration in production endpoint.
- Retrieval-augmented generation (RAG) with document indexing.
- Session memory and personalization using user-scoped Firestore collections.
- Better evaluation with factuality and latency benchmarks.
- Role-based access control and Secret Manager integration for stronger production security.

## Security and Privacy Risks + Mitigation Implemented

Risk 1: - API keys could leak through code commits or verbose logs.

Risk 2: - Tool-calling can expose command-injection paths if unrestricted.

Implemented mitigation: - Input sanitization and strict command allow-list for shell calls. - API keys moved to environment variables and deployment configuration.

## Cross-Cloud or Hybrid Deployment Note

This implementation supports hybrid operation: - Local client (developer machine) sends requests to GCP-hosted assistant service. - Optional secondary LLM provider key can be enabled for cross-provider fallback testing.

## Reproducibility Summary

- Source code structure, environment template, deployment script, and evaluation scripts are included.
- A user with GCP project access can deploy through `scripts/deploy.ps1` and run evaluation with `evaluation/run_eval.py`.